

# Bots and how to catch them

STEM Games 2026

Mathematics arena  $\times$  Erste

Šelini apostoli - Vinko Pirić, Niko Pavlić, Anton Trumbić, Jere Čavala

## 1 Introduction and motivation

In today's digital environment, social media platforms contain not only real users but also a large number of automated accounts. Because of this, it is becoming increasingly important to recognize when an account is controlled by a bot rather than by a human. In this project, we focus on detecting bots on X, formerly known as Twitter. These bots can publish posts, reply to other users, repost content, and sometimes even participate in longer conversations.

We chose this type of bot because X is a platform where information spreads very quickly. A single post can reach thousands or even millions of users in a short time, especially if it is reposted or promoted by many accounts. If some of those accounts are automated, they can create a false impression that a certain opinion, topic, or piece of news is more popular than it really is. This can be especially dangerous in situations involving politics, health, social conflicts, or breaking news.

The real-world impact of such bots can be significant. They may be used to spread misinformation, promote scams, influence public opinion, or increase engagement on controversial content. They can also make online discussions less reliable, because users may believe they are interacting with real people when they are actually reading automatically generated messages.

From a mathematical point of view, bot detection is an interesting problem because bots often behave differently from humans. For example, they may post too frequently, use repetitive language, react very quickly, or interact mostly with a limited group of accounts. These behavioral patterns can be measured and analyzed. Our goal is to use such measurable characteristics to design a method that estimates whether an account is more likely to be human or automated.

## 2 How the bot works

X bots in most cases work on a Rule-based System, while some more advanced bots use Machine learning allowing them to learn from different patterns. The simplest X bots use predefined rules to perform various actions. Examples of rules:

- Respond with a predefined message when bot is tagged,
- Publish a post every certain period (e.g., every hour),
- Repost every tweet with a specific hashtag,
- Follow a list of predefined accounts,
- Like every post that contains a specific keyword,
- Reply to every post that contains a specific keyword with a predefined message.

Rule-based bots are the most simple to code but are easiest to get spotted since they can hardly mimic human behaviour and adapt to new situations. That's why more advanced bots use new technologies such as Machine learning to dodge detection. Machine learning bots are more complex and can learn from data. They can analyze patterns in human behavior and try to mimic them. For example, they can learn when to post, how to interact with other users, and how to generate more natural language. These bots often use algorithms such as decision trees, support vector machines, or neural networks to make decisions based on the data they have been trained on.

The mathematical background of these bots can be quite complex, especially for those that use deep learning. They may involve concepts such as optimization, probability, statistics, and linear algebra. For example, a neural network bot may use backpropagation to adjust its weights based on the error of its predictions, while a rule-based bot may simply follow a set of if-then statements without any learning capability.

### 3 Mathematical analysis

Since bots often have different behavior than humans and are built upon mathematical models, we can analyze their behavior and characteristics to develop a method for detecting them. Here are some of the key indicators that can be used to identify bots on X.

#### 3.1 Account username analysis

One of the first indicators of a suspicious X account is the structure of the username. Bots often generate usernames automatically using random characters and numbers such as: @xj29dk81, @bot784512, @a91bcd77. On the other hand, human usernames are generally more meaningful and contain recognizable words or names.

To measure randomness in usernames, entropy analysis can be used. Entropy measures how unpredictable a sequence of characters is. High entropy usually indicates randomly generated usernames. The Shannon entropy formula is:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

where  $H(X)$  is the entropy of the random variable  $X$ ,  $p(x_i)$  is the probability of the character  $x_i$ , and  $n$  is the number of unique characters in the username. Low entropy indicates that the username is more likely to be human-generated, while high entropy suggests it may be a bot.

### 3.2 Account age and account activity

Bots often have a short lifespan and can post a large number of tweets in a short period. To analyze this, we can calculate the activity coefficient, which is defined as the ratio of the number of posts to the account age:

$$A = \frac{P}{D},$$

where  $A$  is the activity coefficient,  $P$  is the total number of posts, and  $D$  is the account age in days. A high activity coefficient may indicate that the account is a bot, especially if it has been active for only a short time. However, this method has limitations, as some human users may also be very active, and some bots may be designed to post less frequently to avoid detection. Therefore, it is important to use this method in combination with other indicators for a more accurate detection.

### 3.3 Posting frequency analysis

Bots often post at a much higher frequency than humans. They may post every few minutes or even every few seconds, while humans typically post less frequently. To analyze posting frequency, we can calculate the average time interval between posts. If the average interval is very short (e.g., less than 10 minutes), it may indicate bot-like behavior. We can also look at the distribution of posting times. If an account posts at regular intervals (e.g., every hour on the hour), it is more likely to be a bot.

Posting frequency can be analyzed using the following formula:

$$f = \frac{n}{t},$$

where  $f$  is the posting frequency,  $n$  is the number of posts, and  $t$  is the total observed time period. A high value of  $f$  may indicate bot-like behavior.

### 3.4 Variance analysis

Bots often post at regular intervals, which can be detected by analyzing the variance of the time intervals between posts. If the variance is very low, it may indicate that the account is posting at regular intervals, which is a common characteristic of bots. The variance can be calculated using the formula:

$$\text{Var}(\Delta t) = \frac{1}{n} \sum_{i=1}^n (\Delta t_i - \mu)^2,$$

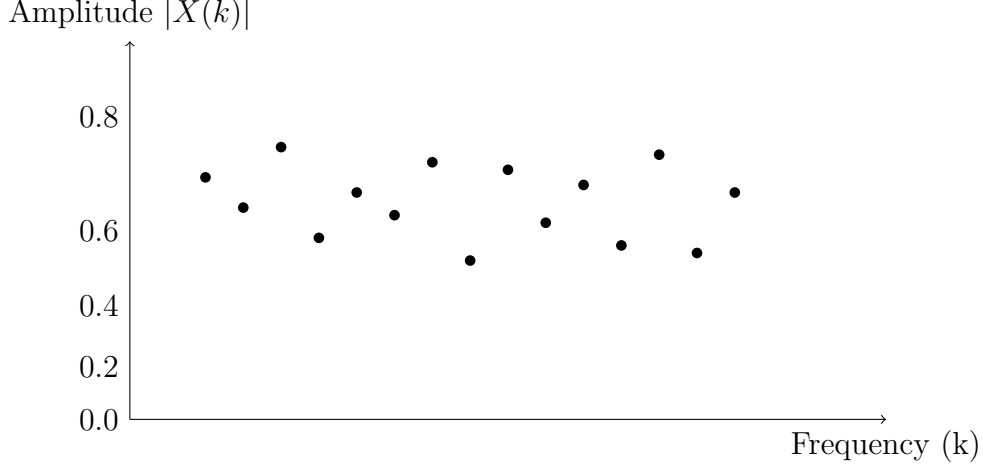
where  $\Delta t_i$  is the time interval between the  $i$ -th and  $(i+1)$ -th post,  $\mu$  is the mean of the time intervals, and  $n$  is the total number of intervals. A low variance suggests that the account may be a bot, while a high variance indicates more human-like behavior.

### 3.5 Fourier analysis

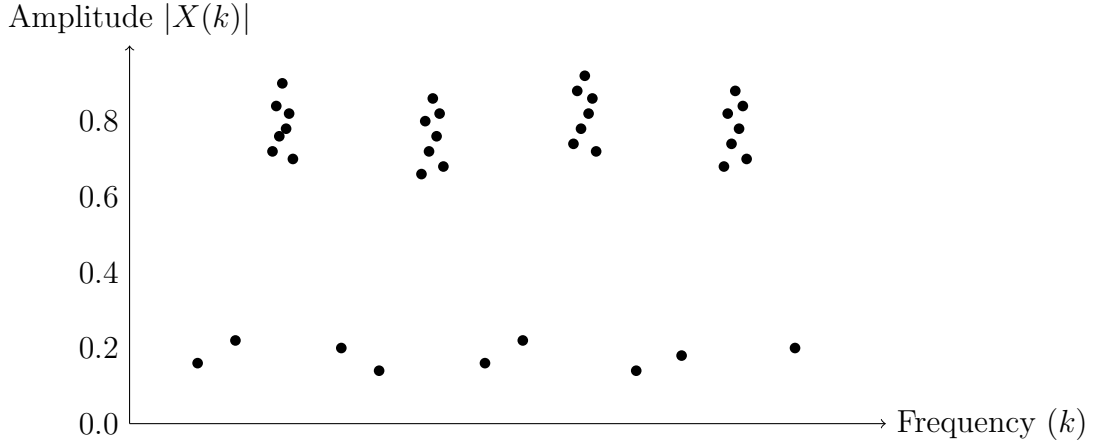
Another method to detect bots is to analyze the periodicity of their posting behavior using Fourier analysis. User activity is modeled as a discrete-time signal and transformed into the frequency domain using the Discrete Fourier Transform (DFT):

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N},$$

where  $x(n)$  is the number of activities (e.g., posts) at time  $n$ ,  $N$  is the total number of time intervals observed, and  $X(k)$  represents the strength of the frequency component at frequency  $k$ . If there are strong peaks in the frequency spectrum, it may indicate that the account is posting at regular intervals, which is a common characteristic of bots. However, this method may not be effective for bots that post irregularly or for human users who have regular posting habits.



Picture 1: Human Activity Spectrum.



Bot Activity Spectrum (Periodic, Strong Peaks)

Picture 2: Fourier spectrum for bot activity.

### 3.6 Kolmogorov–Smirnov test for posting intervals

Another way to analyze bot-like behavior is to compare the distribution of time intervals between posts with a theoretical distribution expected from natural human activity. Human posting behavior is not perfectly periodic. A simple mathematical model for random events in time is the Poisson process. In such a process, the time intervals between consecutive events follow an exponential distribution.

Let the posting times be  $t_1, t_2, \dots, t_n$ . We define the time intervals between consecutive posts as

$$\Delta t_i = t_{i+1} - t_i.$$

If the posts follow a Poisson process, then the intervals  $\Delta t_i$  approximately follow the exponential distribution

$$F(x) = 1 - e^{-\lambda x},$$

where

$$\lambda = \frac{1}{\overline{\Delta t}}$$

and  $\overline{\Delta t}$  is the average time interval between posts.

To measure how different the observed intervals are from the expected exponential distribution, we use the Kolmogorov–Smirnov statistic:

$$D_n = \max_x |F_n(x) - F(x)|,$$

where  $F_n(x)$  is the empirical cumulative distribution function of the observed intervals, and  $F(x)$  is the theoretical exponential cumulative distribution function.

If  $D_n$  is large, the account does not behave like a natural random process. This may indicate automated behavior. For example, a bot that posts every 15 minutes will have almost equal intervals, which is very different from the exponential distribution. Therefore, the Kolmogorov–Smirnov test can detect scheduled or artificially generated posting patterns even when simple frequency analysis is not enough.

### 3.7 Markov chain analysis

An additional mathematical method for detecting bot-like behavior is based on Markov chains. A Markov chain is a stochastic model that describes a system which moves from one state to another, where the next state depends only on the current state and not on the complete history of previous states. This property is called the Markov property and is written as

$$P(X_{n+1} = x_{n+1} \mid X_n = x_n, \dots, X_0 = x_0) = P(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

In the context of social media bot detection, the states can represent different types of user activity. For example, we define the set of states as

$$\mathcal{S} = \{P, R, L, S, I\},$$

where  $P$  represents posting,  $R$  represents replying,  $L$  represents liking,  $S$  represents sharing or reposting, and  $I$  represents inactivity.

The behavior of an account can then be written as a sequence of states:

$$X_1, X_2, \dots, X_n.$$

For example, the sequence

$$P \rightarrow R \rightarrow R \rightarrow S \rightarrow I \rightarrow P$$

means that the account first creates a post, then replies twice, reposts or shares content, becomes inactive for some time, and then posts again.

This approach is useful because human users usually behave less predictably. Their actions depend on interests, emotions, daily routines, and spontaneous reactions to other users. Bots, on the other hand, often follow a limited number of repeated patterns. For example, a bot may repeatedly post, reply, repost, and then become inactive for a fixed period of time. Such behavior can be described mathematically using transition probabilities.

The probability of moving from state  $i$  to state  $j$  is denoted by

$$p_{ij} = P(X_{n+1} = j \mid X_n = i).$$

All transition probabilities form a transition matrix

$$\mathbf{P} = [p_{ij}].$$

For the states  $\{P, R, L, S, I\}$ , the transition matrix has the form

$$\mathbf{P} = \begin{bmatrix} p_{PP} & p_{PR} & p_{PL} & p_{PS} & p_{PI} \\ p_{RP} & p_{RR} & p_{RL} & p_{RS} & p_{RI} \\ p_{LP} & p_{LR} & p_{LL} & p_{LS} & p_{LI} \\ p_{SP} & p_{SR} & p_{SL} & p_{SS} & p_{SI} \\ p_{IP} & p_{IR} & p_{IL} & p_{IS} & p_{II} \end{bmatrix}.$$

The elements of the transition matrix are estimated from observed data. Let  $N_{ij}$  be the number of transitions from state  $i$  to state  $j$ . Then the transition probability is calculated as

$$p_{ij} = \frac{N_{ij}}{\sum_k N_{ik}}.$$

The denominator represents the total number of transitions from state  $i$  to any other state.

For example, if the transition from posting to replying occurs 40 times, and the total number of transitions from posting to any other state is 100, then

$$p_{PR} = \frac{40}{100} = 0.4.$$

This means that, after posting, the account replies with probability 0.4.

A useful measure derived from the transition matrix is transition entropy. It measures how predictable the next action is after a given current action:

$$H_i = - \sum_{j=1}^m p_{ij} \log_2(p_{ij}).$$

If  $H_i$  is low, then the next action is easy to predict, which may indicate automated behavior. If  $H_i$  is high, then the behavior is more diverse and less predictable, which is usually more human-like.

A stronger version of this method is to compare the observed account with two reference models: a human model and a bot model. Let

$$\mathbf{P}_{\text{human}}$$

be the transition matrix learned from human accounts, and let

$$\mathbf{P}_{\text{bot}}$$

be the transition matrix learned from bot accounts.

For an observed sequence of states

$$X_1, X_2, \dots, X_n,$$

we calculate the likelihood that this sequence was generated by the human model:

$$L_{\text{human}} = \prod_{t=1}^{n-1} P_{\text{human}}(X_{t+1} \mid X_t).$$

Similarly, the likelihood that the same sequence was generated by the bot model is

$$L_{\text{bot}} = \prod_{t=1}^{n-1} P_{\text{bot}}(X_{t+1} \mid X_t).$$

Since products of many probabilities can become extremely small, it is better to use logarithms:

$$\begin{aligned} \log L_{\text{human}} &= \sum_{t=1}^{n-1} \log P_{\text{human}}(X_{t+1} \mid X_t), \\ \log L_{\text{bot}} &= \sum_{t=1}^{n-1} \log P_{\text{bot}}(X_{t+1} \mid X_t). \end{aligned}$$

The final comparison is made using the log-likelihood ratio:

$$R = \log L_{\text{bot}} - \log L_{\text{human}}.$$

If

$$R > 0,$$

then the observed sequence is more likely under the bot model. If

$$R < 0,$$

then the observed sequence is more likely under the human model.

This method improves bot detection because it does not analyze only separate numerical features such as posting frequency, variance, or periodicity. Instead, it analyzes the structure of behavior. A human user may sometimes post frequently, but the order of their actions is usually more varied. A bot may post with a similar frequency, but its transitions between activities are often more repetitive and predictable.

Therefore, Markov chain analysis creates a stronger detection barrier. In order to avoid detection, a bot would have to imitate not only human posting frequency and timing, but also the distribution of activities and the transition structure between them. This makes the detector more robust and reduces the probability of false classifications.

### 3.8 Graph theory and matrix representation

Graph theory can also be used for bot detection because social media platforms naturally form networks. Instead of analyzing only one account at a time, we can analyze how accounts interact with each other. Let

$$G = (V, E, \psi_G)$$

be a directed graph, where  $V$  is the set of accounts,  $E$  is the set of interactions, and  $\psi_G$  is an incidence function that assigns each edge to its endpoints. If account  $i$  replies to, reposts, mentions, or likes account  $j$ , we represent this interaction by a directed edge from vertex  $i$  to vertex  $j$ . The direction of the edge indicates the flow of interaction. For example, if account  $A$  mentions account  $B$ , we draw an edge from  $A$  to  $B$ .

This approach is useful because bots often do not act independently. They may form coordinated groups that repost the same content, promote the same user, use the same hashtags, or repeatedly interact with each other. Such coordinated behavior can be detected through the structure of the graph.

The graph can be represented by an adjacency matrix

$$A = [a_{ij}],$$

where

$$a_{ij} = \begin{cases} 1, & \text{if account } i \text{ interacts with account } j, \\ 0, & \text{otherwise.} \end{cases}$$

In a weighted version,  $a_{ij}$  can represent the number of interactions from account  $i$  to account  $j$ . This is useful because repeated interactions between the same accounts can indicate artificial amplification.

From the adjacency matrix, we can calculate the outgoing and incoming degrees:

$$d_{\text{out}}(i) = \sum_j a_{ij},$$

$$d_{\text{in}}(i) = \sum_j a_{ji}.$$

The outgoing degree measures how many interactions account  $i$  sends, while the incoming degree measures how many interactions account  $i$  receives. A suspicious account may have a high outgoing degree and a low incoming degree, meaning that it contacts many users but receives little natural interaction. This can be measured by the ratio

$$R(i) = \frac{d_{\text{out}}(i)}{d_{\text{in}}(i) + 1}.$$

A large value of  $R(i)$  may indicate spam-like or automated behavior. The term  $+1$  is added to avoid division by zero.

The degree matrix is a diagonal matrix whose diagonal elements are the degrees of the vertices:

$$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}.$$

Together with the adjacency matrix, it defines the graph Laplacian:

$$L = D - A.$$

The Laplacian matrix is important because it describes the structure of the graph. It can be used to detect clusters and communities. Bot accounts often form dense artificial communities, where many accounts interact with each other or with the same target account.

The incidence matrix can also be used. If the graph has vertices  $v_1, \dots, v_n$  and edges  $e_1, \dots, e_m$ , then the directed incidence matrix is defined by

$$b_{ij} = \begin{cases} -1, & \text{if edge } e_j \text{ leaves vertex } v_i, \\ 1, & \text{if edge } e_j \text{ enters vertex } v_i, \\ 0, & \text{otherwise.} \end{cases}$$

The incidence matrix describes how interactions flow through the network. If many edges have similar sources and targets, this may indicate coordinated activity.

For example, suppose we have four accounts:

$$V = \{B_1, B_2, B_3, T\},$$

where  $B_1, B_2, B_3$  are suspicious accounts and  $T$  is a target account. Suppose the interactions are

$$B_1 \rightarrow T, \quad B_2 \rightarrow T, \quad B_3 \rightarrow T,$$

$$B_1 \rightarrow B_2, \quad B_2 \rightarrow B_3.$$

This means that all three suspicious accounts promote the same target account, while also interacting with each other.

Using the vertex order

$$(B_1, B_2, B_3, T),$$

the adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The outgoing degrees are

$$d_{\text{out}}(B_1) = 2, \quad d_{\text{out}}(B_2) = 2, \quad d_{\text{out}}(B_3) = 1, \quad d_{\text{out}}(T) = 0.$$

Therefore, the outgoing degree matrix is

$$D_{\text{out}} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The directed Laplacian is then

$$L = D_{\text{out}} - A = \begin{bmatrix} 2 & -1 & 0 & -1 \\ 0 & 2 & -1 & -1 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

We can also calculate the outgoing-to-incoming ratio. For account  $B_1$ , we have

$$d_{\text{out}}(B_1) = 2,$$

and since no other account interacts with  $B_1$ ,

$$d_{\text{in}}(B_1) = 0.$$

Therefore,

$$R(B_1) = \frac{2}{0+1} = 2.$$

For the target account  $T$ , we have

$$d_{\text{out}}(T) = 0,$$

and

$$d_{\text{in}}(T) = 3.$$

This shows that  $T$  receives many interactions but does not send any. If many low-quality or suspicious accounts point to the same target, this can indicate artificial amplification.

For the same graph, define the edges as

$$e_1 = B_1 \rightarrow T, \quad e_2 = B_2 \rightarrow T, \quad e_3 = B_3 \rightarrow T,$$

$$e_4 = B_1 \rightarrow B_2, \quad e_5 = B_2 \rightarrow B_3.$$

The directed incidence matrix is

$$B = \begin{bmatrix} -1 & 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 & -1 \\ 0 & 0 & -1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}.$$

Each column represents one interaction. The value  $-1$  marks the account where the interaction starts, and  $1$  marks the account where the interaction ends.

This example shows why graph theory is useful for bot detection. Even if each bot tries to imitate human timing, the group may still create a suspicious network structure. Many accounts may point to the same target, interact with each other, or form dense communities. Therefore, adjacency matrices, degree matrices, incidence matrices, and Laplacian matrices can reveal coordinated behavior that would not be visible by analyzing only one account.

### 3.9 Multidimensional scoring model

The main idea behind a multidimensional scoring model is that all metrics are combined into a single function and analyzed together. A single metric is usually not sufficient for reliable classification because it may produce false positives or false negatives. For example, a journalist may post very frequently during breaking news events, while a streamer or influencer may naturally create many posts per day. Therefore, it is necessary to observe multiple behavioral indicators simultaneously.

A general form of the bot scoring function can be written as

$$B = f(E, A, F, V, P, \dots),$$

where

$B$  = bot score,

$E$  = entropy,

$A$  = activity coefficient,

$F$  = posting frequency,

$V$  = variance,

and

$P$  = periodicity or Fourier score.

The simplest implementation is a linear combination of metrics:

$$B = w_1 E + w_2 A + w_3 F + w_4 V + w_5 P,$$

where

$$w_1, w_2, \dots, w_n$$

are weighting coefficients. The weights are chosen according to the importance and reliability of each metric. For example, posting frequency alone may not be as reliable as periodicity detection or Markov-chain entropy analysis.

This approach means that the decision is not based on one isolated value. Instead, every account is represented as a vector

$$x = (E, A, F, V, P),$$

which corresponds to a point in a multidimensional feature space. Human users and bot accounts can then form different clusters in this space. Human accounts usually occupy more irregular regions, while bots tend to group together because their behavior is more repetitive and predictable.

An important step in this process is normalization. Without normalization, metrics with large numerical values may dominate the final score. A common normalization method is standardization:

$$x' = \frac{x - \mu}{\sigma},$$

where  $\mu$  is the mean value and  $\sigma$  is the standard deviation.

After normalization, the final bot score is compared with a classification threshold  $T$ . If

$$B > T,$$

the account is classified as a bot. Otherwise, if

$$B \leq T,$$

the account is classified as a human user.

This multidimensional approach significantly improves robustness because a bot must imitate several independent behavioral properties simultaneously. Even if a bot successfully hides one suspicious feature, such as posting frequency, it may still be detected through periodicity, entropy, variance, or transition behavior. Therefore, this method creates a stronger detection barrier and reduces the likelihood of false classifications.

## 4 Implementation

The Twitter/X bot detection system was implemented in the C programming language. The program uses a modular structure, where each function analyzes one specific property of the account and contributes to the final bot score. The main goal of the implementation is to combine statistical and time-based indicators in order to classify an account as human-like or bot-like.

The system uses a custom structure called `TwitterAccount`, which contains the most important account attributes:

- username,
- account creation date,
- number of messages,
- array of posting timestamps.

These parameters are used for all mathematical calculations in the detection process.

### 4.1 Account simulation

To test the detector, the program generates 1000 simulated Twitter/X accounts. The accounts are divided into two main groups: human-like accounts and bot-like accounts.

Human-like accounts are generated with irregular posting intervals, lower posting frequency, older account ages, more natural usernames, and higher randomness in posting behavior. Bot-like accounts are generated with periodic posting schedules, higher posting frequency, randomly generated usernames, lower interval variance, and more recently created accounts.

This simulation allows large-scale testing without using external APIs or real user data.

## 4.2 Implemented detection methods

The implementation combines several mathematical criteria. Username entropy is calculated using Shannon entropy in order to detect randomly generated usernames. Higher entropy suggests a more random character distribution and therefore increases the bot suspicion score.

The activity coefficient measures how active an account is relative to its age:

$$A = \frac{\text{number of posts}}{\text{account age in days}}.$$

Accounts with many posts in a short period receive a higher score.

Posting frequency is calculated as the number of posts per hour. This detects accounts that publish content at unusually high rates. Variance analysis is used to measure the regularity of posting intervals. Human users usually post at irregular times, while bots often have very low variance between consecutive posts.

The system also includes periodicity detection. This module compares neighboring time intervals between posts. If many intervals are almost equal, the account receives a higher periodicity score because repeated timing patterns are typical for automation. Finally, Fourier analysis is implemented to detect strong peaks in the frequency spectrum of posting behavior. This is done by applying the Discrete Fourier Transform to the time series of posting activity. Accounts with strong periodic components receive a higher bot score.

## 4.3 Bot Scoring System

The proposed bot detection model uses a multidimensional scoring system. Instead of relying on only one metric, several behavioral and structural features are combined into a single bot score. This is important because a single metric can produce false positives or false negatives. For example, a journalist may post very frequently during a short event, while a streamer may naturally have several posts per day. Therefore, the final decision is based on a combination of username structure, activity intensity, posting frequency, interval regularity, periodicity, and Fourier-based periodic behavior.

The total bot score is defined as

$$B = S_E + S_A + S_F + S_V + S_P + S_D,$$

where  $B$  is the final bot score, and the individual components are:

$S_E$  – entropy score,

$S_A$  – activity score,

$S_F$  – posting frequency score,

$S_V$  – interval variance score,

$S_P$  – periodicity score,

$S_D$  – Fourier/DFT score.

The maximum possible value of  $B$  is 100 points. A higher score indicates a higher probability that the account shows automated behavior.

| Metric                               | Maximum score |
|--------------------------------------|---------------|
| Username entropy $S_E$               | 10            |
| Activity coefficient $S_A$           | 20            |
| Posting frequency $S_F$              | 20            |
| Interval regularity / variance $S_V$ | 15            |
| Periodicity ratio $S_P$              | 20            |
| Fourier peak ratio $S_D$             | 15            |
| <b>Total</b>                         | <b>100</b>    |

Table 1: Weights of the bot scoring system

## 5 Evaluation

The evaluation was performed on a dataset of 700 accounts, consisting of 427 human-like accounts and 273 bot-like accounts. Accounts classified as MEDIUM RISK or HIGH RISK were treated as detected bot accounts, while accounts classified as LOW RISK were treated as human accounts.

The model correctly classified 562 out of 700 accounts, resulting in an overall accuracy of 80.29%. There were no false positive classifications, meaning that no human-like accounts were incorrectly classified as bots. However, 138 bot-like accounts were classified as human accounts, which resulted in a recall of 49.45

These results show that the model is conservative: it avoids incorrectly labeling human users as bots, but it misses some less obvious bot-like accounts. This indicates that the scoring thresholds could be adjusted if the goal is to detect a larger number of suspicious accounts.

| Measure                         | Number of accounts | Percentage |
|---------------------------------|--------------------|------------|
| True Positives (TP)             | 135                | 19.29%     |
| True Negatives (TN)             | 427                | 61.00%     |
| False Positives (FP)            | 0                  | 0%         |
| False Negatives (FN)            | 138                | 19.71%     |
| Correctly classified accounts   | 562                | 80.29%     |
| Incorrectly classified accounts | 138                | 19.71%     |

Table 2: Evaluation results of the bot detection model

|                      | Predicted: bot | Predicted: human |
|----------------------|----------------|------------------|
| <b>Actual: bot</b>   | 135            | 138              |
| <b>Actual: human</b> | 0              | 427              |

Table 3: Confusion matrix

Here,  $TP$  represents correctly detected bots,  $TN$  represents correctly detected human users,  $FP$  represents human users incorrectly classified as bots, and  $FN$  represents bots incorrectly classified as human users.

From these values, we calculate standard evaluation metrics:

| Metric      | Value   |
|-------------|---------|
| Accuracy    | 80.29%  |
| Precision   | 100.00% |
| Recall      | 49.45%  |
| Specificity | 100.00% |
| F1-score    | 66.18%  |

Table 4: Evaluation metrics

Accuracy measures the total percentage of correct classifications. Precision measures how many accounts classified as bots were actually bots. Recall measures how many real bots were successfully detected. The  $F_1$ -score combines precision and recall into one metric.

## 6 Limitations

Although the proposed detector combines several mathematical methods, it still has limitations. First, the implementation is based on simulated data, which means that real social media behavior may be more complex and less predictable. Real users may sometimes behave similarly to bots, especially journalists, influencers, streamers, or customer support accounts that post frequently during short time periods.

Second, advanced bots may intentionally avoid simple detection methods. For example, a bot can add random delays between posts, use more realistic usernames, reduce posting frequency, or imitate human-like daily activity. This can reduce the effectiveness of frequency, variance, and periodicity-based methods.

Third, some advanced methods described in the mathematical analysis, such as Markov-chain comparison, graph-based detection, and Kolmogorov–Smirnov testing, require more data than simple account-level statistics. In real deployment, the system would need access to interaction data, post content, timestamps, and possibly labeled training examples.

Therefore, the proposed detector should be understood as a risk estimation system, not as absolute proof that an account is a bot. The final decision should be based on multiple indicators and, in sensitive cases, human review.

### 6.1 False positives and false negatives

A false positive occurs when a real human account is classified as a bot. This may happen when a user has unusually high activity, such as a journalist posting during breaking news, a streamer promoting an event, or a customer support account replying to many users.

A false negative occurs when a bot is classified as a human. This may happen when a bot intentionally adds randomness to its posting schedule, uses a realistic username, avoids repeated messages, or imitates human-like interaction patterns.

The purpose of combining multiple metrics is to reduce both types of errors. A single feature, such as high posting frequency, is not reliable enough on its own. However, if high frequency appears together with low variance, strong Fourier peaks, low Markov entropy, and suspicious graph structure, the probability of bot-like behavior becomes much higher.

## 7 Further development

Future development of the Twitter/X bot detection system could significantly improve accuracy by introducing advanced behavioral, metadata, multimedia, and network analysis

techniques. The current system is mostly based on mathematical and behavioral indicators, but future versions could include additional modules that analyze images, metadata, natural language, and coordinated activity between accounts.

## 7.1 Image metadata analysis

One important upgrade would be image metadata verification from posted images and media attachments. Many automated bot networks reuse generated, stolen, or manipulated images. By analyzing image metadata, the system could detect suspicious patterns that are not visible from text or timing data alone.

The system could inspect:

- EXIF metadata,
- camera and device information,
- GPS coordinates,
- image creation timestamps,
- editing software signatures,
- duplicate image hashes,
- compression artifacts.

Bots frequently use:

- AI-generated profile pictures,
- stock photos,
- reuploaded images,
- mass-generated propaganda media.

Some suspicious metadata indicators are shown in Table 5.

| Metadata signal                 | Possible meaning                         |
|---------------------------------|------------------------------------------|
| Missing EXIF data               | Artificially generated or stripped image |
| Same device ID on many accounts | Coordinated bot farm                     |
| Identical timestamps            | Automated uploads                        |
| Repeated image hashes           | Spam reposting                           |
| Photoshop or AI software traces | Synthetic or edited content              |

Table 5: Examples of suspicious image metadata signals.

## 7.2 Reverse image detection

A future module could compare uploaded images against previously known suspicious images. This could be done using reverse image search APIs, public spam databases, or internal databases of previously detected bot accounts.

This would help identify:

- fake profile pictures,
- deepfakes,
- reused propaganda images,
- coordinated misinformation campaigns,
- stolen or stock images used by multiple accounts.

If the same image appears across many unrelated accounts, especially with similar captions or hashtags, this may indicate coordinated bot activity.

## 7.3 AI-generated image detection

Modern bots increasingly use AI-generated avatars and synthetic media. Future versions of the detector could integrate image-forensic methods for detecting artificial content.

Possible techniques include:

- GAN artifact detection,
- diffusion model fingerprinting,
- face asymmetry analysis,
- eye and reflection inconsistency detection,
- unnatural texture or background pattern detection.

This would be especially useful for detecting bot accounts that try to appear realistic by using AI-generated profile pictures.

## 7.4 Multimedia correlation engine

Another important future improvement would be a multimedia correlation engine. Instead of analyzing each signal separately, the system could combine several types of information.

The detector could correlate:

- posting times,
- shared image fingerprints,
- caption similarity,
- hashtag repetition,
- URL repetition,
- network interaction graphs.

This would allow the system to detect entire bot clusters instead of only individual suspicious accounts. For example, if several accounts post the same image, use similar captions, share the same link, and interact with the same target account, the system could classify them as a coordinated group.

## 7.5 Advanced future features

Additional future improvements could include:

- machine learning classification,
- neural network behavioral prediction,
- natural language processing for semantic similarity analysis,
- retweet and repost graph analysis,
- community anomaly detection,
- real-time streaming detection,
- Fourier periodicity analysis,
- Markov-chain behavioral modeling,
- graph-based detection of coordinated communities.

These techniques would make the detector more flexible and more resistant to changes in bot behavior. Since bot developers constantly adapt their methods, a future detection system should also be adaptive and regularly updated with new data.

## 7.6 Long-term goal

The long-term goal is to create a hybrid detection engine that combines:

- mathematical analysis,
- time-series statistics,
- metadata verification,
- image forensics,
- natural language processing,
- social graph intelligence,
- machine learning prediction.

Such a system would approach the architecture used in modern large-scale anti-bot and anti-disinformation platforms. By combining several independent detection methods, the system would become more robust. A bot may be able to hide one suspicious feature, such as posting frequency, but it is much harder to imitate human behavior across timing, content, metadata, images, and network structure at the same time.

## 8 Conclusion

This paper presented a multidimensional approach for detecting bot-like behavior on social media accounts. Instead of relying on a single feature, the proposed model combines several indicators, including username entropy, account activity, posting frequency, interval variance, periodicity, and Fourier-based frequency analysis. By combining these metrics into one bot score, the system provides a more reliable evaluation of suspicious account behavior.

The implementation was tested on a dataset of 700 generated accounts containing both human-like and bot-like activity patterns. The results show that the model is conservative, meaning that it avoids incorrectly classifying human users as bots. However, some less obvious bot-like accounts were still classified as human accounts, which shows that there is room for improvement. Future improvements could include using larger real-world datasets, adding network-based features, analyzing post content, and applying machine learning methods to automatically adjust scoring thresholds.

Overall, the project demonstrates that behavioral and temporal patterns can be useful indicators for detecting automated accounts, especially when multiple metrics are combined into a single scoring system.

## 9 References

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] Emilio Ferrara, Onur Varol, Clayton Davis, Filippo Menczer, and Alessandro Flammini. The rise of social bots. <https://dl.acm.org/doi/10.1145/2818717>, 2016. Accessed: 2026-05-19.
- [3] Observatory on Social Media. Botometer x. <https://botometer.osome.iu.edu/>, 2026. Accessed: 2026-05-19.
- [4] Alan V. Oppenheim, Ronald W. Schaffer, and John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 1999.